

ECE 268 Project Progress Report

Lightweight Block Ciphers for Constrained Devices: PRESENT-80 and SPECK-64

Devansh Gupta

Priyansh Parikh

Jeevan N V

May 21, 2026

Project Summary

Goal: Implement PRESENT-80 and SPECK-64 from scratch with encryption, decryption, and key schedules. Each cipher will be wrapped in CBC and CTR modes, validated against published test vectors, and benchmarked on GPU against a sequential CPU baseline and AES-128, with IoT/RFID as the target setting where AES is too expensive.

Status: PRESENT-80 is complete on both CPU and GPU. All four official test vectors pass and GPU performance has been measured. SPECK-64, CBC/CTR modes, and the AES baseline are pending.

1 Work Completed So Far

Implementations

- **Repository:** Current files are `present80.cpp`, `present80.cu`, and `CMakeLists.txt`. The planned files `speck64.cpp`, `speck64.cu`, `modes.cpp`, and an AES baseline are not yet written.
- **PRESENT-80 CPU:** 64-bit block, 80-bit key, 32 rounds per Bogdanov et al. The key schedule rotates the 80-bit register left by 61, applies the S-box to the top nibble, and XORs the round counter into bits 19–15. Encryption runs 31 full rounds of `addRoundKey`, `sBoxLayer`, and `pLayer`, then adds the final round key. Decryption reverses those steps using `INV_SBOX` and the inverse P-layer. The bit permutation formula is $p(j) = (16j) \bmod 63$ with bit 63 fixed in place.
- **PRESENT-80 GPU:** A CUDA port of the CPU implementation. Round keys and S-box tables go into `__constant__` memory so they are broadcast across each warp without extra traffic. The encrypt and decrypt kernels each assign one thread per block. The P-layer helper is `__device__ __forceinline__` with `#pragma unroll` on the 63-iteration loop. Key expansion runs on the CPU and the result is copied to constant memory before the kernel launches. The test bench encrypts and then decrypts 2^{20} distinct blocks and verifies that every output matches the original plaintext.
- **Build system:** `CMakeLists.txt` uses a `USE_CUDA` flag. ON builds `present80.cu` with `nvcc` targeting the native GPU architecture. OFF builds `present80.cpp` as a C++17 binary with `-O4`.

Experiments and Evaluations

- **Setup:** Test vectors are the four canonical cases from Bogdanov et al. GPU timing uses CUDA events over 2^{20} blocks on an NVIDIA RTX 4070 Mobile running CUDA 12.0. CPU timing uses `std::chrono::high_resolution_clock` on the same machine at ≈ 3.8 GHz.
- **Completed:** All four PRESENT-80 spec vectors pass on the CPU build. GPU round-trip correctness was verified across all 2^{20} blocks. CPU and GPU encrypt throughput were both measured on the same workload.
- **Pending:** SPECK-64 and CBC/CTR tests have not been run because those components are not yet implemented.

Preliminary Results

PRESENT-80 passes all published vectors. For the same 2^{20} -block workload the GPU finishes in 7.85 ms versus 12,518 ms on the CPU, a roughly $1595\times$ speedup. The CPU implementation is intentionally unoptimized and

Cipher / Platform	Block/Key	Enc/Dec	Spec Vectors	Throughput
PRESENT-80, CPU sequential	64 / 80	Round-trip OK	4/4 Pass	≈ 0.67 MB/s
PRESENT-80, GPU RTX 4070M	64 / 80	Round-trip OK	4/4 Pass	1.07 GB/s enc, 1.43 GB/s dec
SPECK-64, CPU	64 / —	—	—	Not started
SPECK-64, GPU	64 / —	—	—	Not started
AES-128 baseline	128 / 128	—	—	Pending

sits at ≈ 4300 cycles/byte.

2 Challenges and Issues Encountered

- **Key-schedule bit ordering:** Loading the 80-bit key into `std::bitset<80>` requires careful mapping because key byte 0 is the most significant byte. An off-by-one in the bit indexing produces wrong round keys that still pass the round-trip test, so the error only surfaces when running the official vectors.
- **Round key extraction:** The spec takes the leftmost 64 bits of the 80-bit register as the round key. Getting the MSB/LSB alignment right between the `bitset` and the `uint64_t` required cross-checking against each test vector.
- **P-layer boundary case:** The CPU version handles the $j = 63$ fixed point with a ternary inside a 0-to-63 loop. The GPU port splits this into a 0-to-62 loop with a separate bit-63 check, which is easier to verify.
- **CMake and CUDA:** `CMAKE_CUDA_COMPILER` is not resolved automatically in all environments. The GPU build currently requires either the `CUDACXX` environment variable or calling `nvcc` directly.
- **Scope:** PRESENT-80 uses a 64-bit block with an 80-bit key and 32 rounds. For SPECK-64, the key width needs to be confirmed from the SIMON/SPECK appendix before implementation starts. Gate-count analysis is out of scope and the PRESENT reference value of 1570 GE will only be cited for context.

3 Plans for the Next Two Weeks

Week 1 (May 22–28): Implement SPECK-64 on CPU and GPU from the NSA spec. Validate against the published appendix vectors. Write CBC and CTR mode wrappers that work with both ciphers and verify round-trip correctness for each.

Week 2 (May 29–June 4): Add an AES-128 baseline using OpenSSL. Run benchmarks covering GB/s, cycles/byte, and key-schedule cost at 1 KB, 16 KB, and 64 KB payloads. Compile the comparison table and finish the report and slides.

Planned evaluations: Full vector sweep for SPECK-64 and both mode wrappers. Median GB/s over 10 GPU runs. GPU vs. CPU speedup. `.text` size comparison across PRESENT-80, SPECK-64, and AES-128. A Feistel vs. SPN trade-off discussion for constrained devices.

Presentation (June 4): 10–12 slides on cipher architectures, GPU kernel design choices, benchmark results, and lightweight-crypto analysis. Slides will be uploaded to the shared Drive before the session.

Member	Tasks
Devansh Gupta	SPECK-64 CPU/GPU implementation, AES-128 baseline, results plots
Priyansh Parikh	CBC/CTR mode wrappers, benchmark scripts, PRESENT-80 test-vector harness
Jeevan N V	SPECK-64 vector validation, build cleanup, slides

Feasibility and Direction

PRESENT-80 is done and SPECK-64 is the immediate critical path. One to two days of implementation and one day of debugging is a reasonable estimate. CBC/CTR wrappers are straightforward once both ciphers pass vectors. If a custom CUDA AES takes too long, OpenSSL provides a ready-made reference for the baseline.

The final deliverables are passing test vectors for both ciphers in all modes, a GPU vs. CPU comparison table, and a lightweight-crypto analysis for the presentation.

References: Bogdanov et al., PRESENT (CHES 2007). Beaulieu et al., SIMON/SPECK (ePrint 2013/404).